

Assignment - 1

Aim: Use sequence data types and their associated operations in Python.

Theory: Python offers two fundamental data structures for managing collections of items: lists and sets. While both serve to store multiple elements, their operational characteristics and ideal applications vary significantly. A list is characterized by its ordered nature and mutability, accommodating duplicate entries. Conversely, a set is an unordered and mutable collection, strictly enforcing uniqueness among its elements.

A primary distinction lies in their handling of element sequence. Lists preserve the insertion order, allowing elements to be retrieved by their positional index. Sets, however, do not guarantee any specific order, thus precluding access via indexing.

The management of duplicate values presents another critical divergence. Lists permit the inclusion of identical elements, making them apt for scenarios where repeated data is permissible or necessary. Sets, on the other hand, inherently filter out any duplicate items, storing each element only once, which is invaluable for ensuring data integrity and uniqueness.

Regarding their modifiability, both lists and sets can be altered post-creation. Yet, sets uniquely enforce the condition that all their members must be distinct, thereby ensuring the absence of redundancies within the collection.

```
text = "Python"
print(text[0])
print(text[3])
print(text[-1])
my_list = [10, 20, 30, 40, 50, 60]
print(my_list[1:5:2])

data = [1, 2, 'hello', 4.5]
print("List:", data)

import array
arr = array.array('i', [1, 2, 3, 4])
print("Array:", arr)

text = "DataScience"
print("String:", text)
print("Length of string:", len(text))
print("First three characters:", text[:3])

numbers = [12, 45, 7, 23, 56]
print("\nList:", numbers)
print("Maximum value:", max(numbers))
print("Minimum value:", min(numbers))

student_dict = {19: "Sarang", 22: "Amit", 33: "Rohit", 44: "Neha"}
print("\nDictionary:", student_dict)
print("Roll Numbers (Keys):")
for key in student_dict.keys():
    print(key)

list_numbers = [1, 2, 3, 4, 2, 3, 5, 6, 1]
print("\nOriginal List:", list_numbers)
unique_set = set(list_numbers)
print("Set (Duplicates removed):", unique_set)

print("\nNumbers using range (1 to 5):")
for i in range(1, 6):
    print(i)
```

```
P
h
n
[20, 40]
List: [1, 2, 'hello', 4.5]
Array: array('i', [1, 2, 3, 4])
String: DataScience
Length of string: 11
First three characters: Dat

List: [12, 45, 7, 23, 56]
Maximum value: 56
Minimum value: 7

Dictionary: {19: 'Sarang', 22: 'Amit', 33: 'Rohit', 44: 'Neha'}
Roll Numbers (Keys):
19
22
33
```

44

Original List: [1, 2, 3, 4, 2, 3, 5, 6, 1]
Set (Duplicates removed): {1, 2, 3, 4, 5, 6}

Numbers using range (1 to 5):

1
2
3
4
5

```
import pandas as pd
import numpy as np

# Create a sample DataFrame with missing values for demonstration
data = {
    'Name': ['Sarang', 'Amit', 'Rohit', 'Neha', None],
    'Age': [20, 22, np.nan, 21, 23],
    'Score': [85, np.nan, 90, 88, 75]
}
df = pd.DataFrame(data)

print("Original DataFrame:")
print(df)

print("\nIs Null Check:")
print(df.isnull().sum())

# 4d: Fill missing Age with the mean Age (Data Wrangling step)
df['Age'] = df['Age'].fillna(df['Age'].mean())

# 4c: Drop rows where Name is missing
df = df.dropna(subset=['Name'])

# 5: Column Transformation (Example: Adding a 'Status' column)
# If score > 80, Status is 'Pass', else 'Average'
df['Status'] = df['Score'].apply(lambda x: 'Pass' if x > 80 else 'Average')

print("\nWrangled DataFrame:")
print(df)
```

Original DataFrame:

	Name	Age	Score
0	Sarang	20.0	85.0
1	Amit	22.0	NaN
2	Rohit	NaN	90.0
3	Neha	21.0	88.0
4	None	23.0	75.0

Is Null Check:

Name	1
Age	1
Score	1

dtype: int64

Wrangled DataFrame:

	Name	Age	Score	Status
0	Sarang	20.0	85.0	Pass
1	Amit	22.0	NaN	Average
2	Rohit	21.5	90.0	Pass
3	Neha	21.0	88.0	Pass

1. Sequence Data Types in Python

Sequence data types are data structures that store multiple elements in an ordered manner. Each element is assigned a position (index) which allows accessing elements individually.

- Main Sequence Data Types:

1. String (str): A sequence of characters.
2. List (list): An ordered, mutable collection of elements.
3. Tuple (tuple): An ordered, immutable collection of elements.
4. Range (range): An immutable sequence of numbers, often used in loops.

- *Indexing and Slicing:*

1. Indexing: Accessing individual elements using their position starting at 0 Negative indexing accesses elements from the end (e.g., -1).
2. Slicing: Extracting a subsequence using sequence[start:end:step].

3. start: Starting index (inclusive).
4. end: Ending index (exclusive).
5. step: Gap between elements (defaults to 1).

2. Why prefer tuples?

Tuples in Python represent an ordered, unchangeable collection of items. This immutability means that once a tuple is established, its constituent values cannot be altered. They are particularly valuable in situations where data needs to remain constant and protected from accidental modification.

Reasons to Prefer Tuples:

- **Ensuring Data Reliability:** Given their immutable nature, the contents of tuples cannot be changed after their initial creation. This characteristic guarantees the stability and consistency of the data throughout the program's execution.
- **Enhanced Performance:** Compared to lists, tuples offer superior memory efficiency and quicker processing. This is because they are designed without support for dynamic alterations such as adding or removing elements, leading to a streamlined structure.
- **Suitability as Dictionary Keys:** Due to their immutability and hashability, tuples are perfectly suited for use as keys within dictionaries. In contrast, lists, being mutable, cannot fulfill this role.

3. Python Arrays vs Lists

Definition: Both arrays and lists in Python serve the purpose of holding collections of data. Nevertheless, Python's native `list` structure is versatile, capable of storing items of diverse data types. In contrast, arrays (typically from external libraries like NumPy or Python's `array` module) are specialized for storing homogeneous data types, leading to more optimized storage and processing.

Key Distinctions Between Arrays and Lists

- **Heterogeneity vs. Homogeneity:** Lists can accommodate a mixture of data types, such as integers, strings, or floats within a single collection. Arrays, by design, are restricted to storing elements of a uniform data type.
- **Memory Footprint:** Arrays are generally more memory-efficient due to their compact internal representation. Lists, owing to their inherent flexibility, tend to consume more memory.
- **Computational Speed:** For tasks involving numerical computations, arrays—especially when backed by libraries like NumPy—demonstrate superior performance. Lists, conversely, are less efficient for such computationally intensive operations.
- **Operational Scope:** Lists come equipped with a broader set of built-in methods, making them suitable for general programming tasks. Arrays, on the other hand, are primarily geared towards specialized mathematical and numerical applications.
- **Simplicity of Adoption:** Lists are straightforward to use and do not necessitate the import of any external modules. Arrays frequently require importing specific modules like `array` or `NumPy` before they can be utilized.

4. Internal Working of Dictionaries

Python dictionaries function as unordered collections, associating unique keys with their respective values. At their core, dictionaries leverage a hash table implementation to achieve highly efficient data storage and retrieval.

- **How Dictionaries Operate (The Hash Table Principle):**
 1. **Hashing Process:** Upon the insertion of a key into a dictionary, Python employs its intrinsic `hash()` function to transform the key into an integer, known as a hash value.
 2. **Allocation to Buckets:** This computed hash value then dictates the specific memory location (or 'bucket') where the key-value pair will reside. This method facilitates direct placement rather than requiring sequential scanning.
 3. **Rapid Data Access (O(1) Complexity):** For retrieving a value, Python re-calculates the key's hash and proceeds directly to the designated memory address. This mechanism ensures constant time complexity for operations such as searching, adding, and removing elements, denoted as O(1).

5. Sets vs Lists

Python provides both lists and sets as mechanisms for holding collections of items. A list is an ordered, modifiable collection that permits duplicate entries, whereas a set is an unordered collection exclusively designed to hold distinct, unique elements.

- **Contrasting Features of Sets and Lists:**
 1. **Element Sequence:** Lists preserve the sequential order in which elements are added, enabling access by position. Sets, conversely, do not impose or maintain any particular order among their elements.
 2. **Duplicate Handling:** Lists are tolerant of identical values, allowing multiple instances of the same element. Sets, by nature, automatically eliminate any repeated elements, ensuring that each member is unique.

3. Modifiability: Both lists and sets are mutable, meaning their contents can be altered after creation. However, a key aspect of sets' mutability is their strict enforcement against the inclusion of duplicate items.

Conclusion:

To summarize, Python's sets and lists fulfill distinct roles within programming. Lists are optimally suited for scenarios where the preservation of element order and the allowance of duplicate values are crucial. In contrast, sets are the go-to choice when the primary requirements are unique elements and expedited operational performance. A clear comprehension of these fundamental differences is vital for selecting the most appropriate data structure for a given problem, ultimately fostering more streamlined and robust code development.